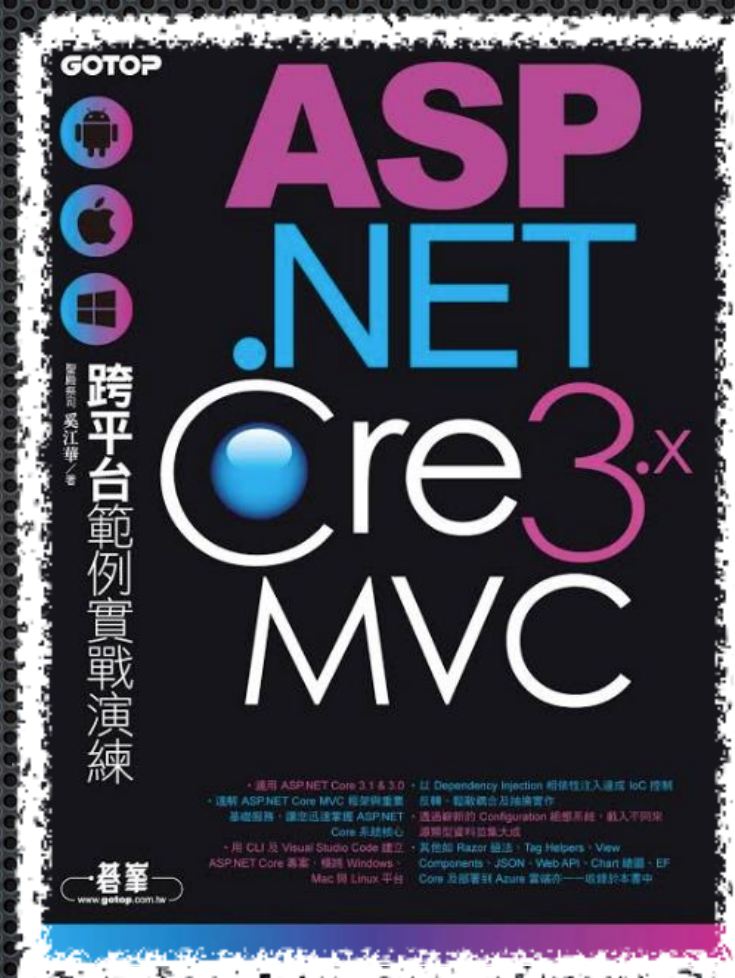


第16章

EF Core Code First 程式優先、 DbContext 與 CLI 命令工具



作者：聖殿祭司。奚江華

大綱

16-1 什麼是 Code First 程式優先

16-2 使用 Code First 及 EF Migrations 建立部落格程式與資料庫

16-3 DbContext 與 DbSet<TEntity> 功用

16-4 DbContext 調用與 DbContextOptions 設定資料庫
Provider 及連線

16-5 使用 Sqlite、MySQL 及 In-Memory 提供者跨資料庫平台

16-6 EF Core 的 CLI 命令工具

什麼是 Code First 程式優先

16-1

Code First和其他優先之對比

- 若說Database First 和 Model First的模型設計必須仰賴EF提供的視覺化工具
- 那麼對Code First來說，便徹底揚棄了對 EF視覺化設計工具的使用，以手工建立程式

Code First程式優先的精神

以 POCO (Plain Old C# Object) 方式建立：

- Entity類別
- DbContext類別
- DbSet<T>實體集
- Navigation Property 導覽屬性
- Association 關聯等
- 搭配一些 Code First 獨有機制

三種開發模式之適用比較

- Database First適合既有存在的資料
- Model First適合用來建立全新的資料庫
- Code First好處是同時適用兩者，無論是既有資料庫，或建立全新的資料庫

	Database First	Model First	Code First
優先主義	資料庫優先	模型優先	程式優先
針對資料庫場景	既有資料庫	新建資料庫	- 既有資料庫 - 新資料庫
模型建立方向	由既有資料庫 導出模型	由模型建立新 資料庫	- 由既有資料庫導出模型 - 由模型建立新資料庫
建立.edmx 檔	✓	✓	
EDM 視覺化工具	✓	✓	
模型定義格式	XML	XML	Class 類別
Entity Class	工具自動產生	工具自動產生	手工撰寫/工具產生
DbContext 類別	工具自動產生	工具自動產生	手工撰寫/工具產生
DbSet<T>	工具自動產生	工具自動產生	手工撰寫/工具產生
Migrations 資料庫			✓
樣本資料佈建機制			✓

Code First模型的Model 和 DbContext 類別建立

- 若是建立新資料庫，Model 和 DbContext 類別必須先建立，Migration才能據以產出資料庫
- 若是既有資料庫，除了上面方式，亦提供工具支援 Scaffolding 產出

使用 Code First 及 EF Migrations 建立部落格程式與資料庫

16-2

Blog實體與關聯

- 在此以一個部落格應用程式為例，首先建立 User（使用者）、Blog（部落格）和 Post（貼文）三個 Entities 實體，以及三者間的 Association 關聯



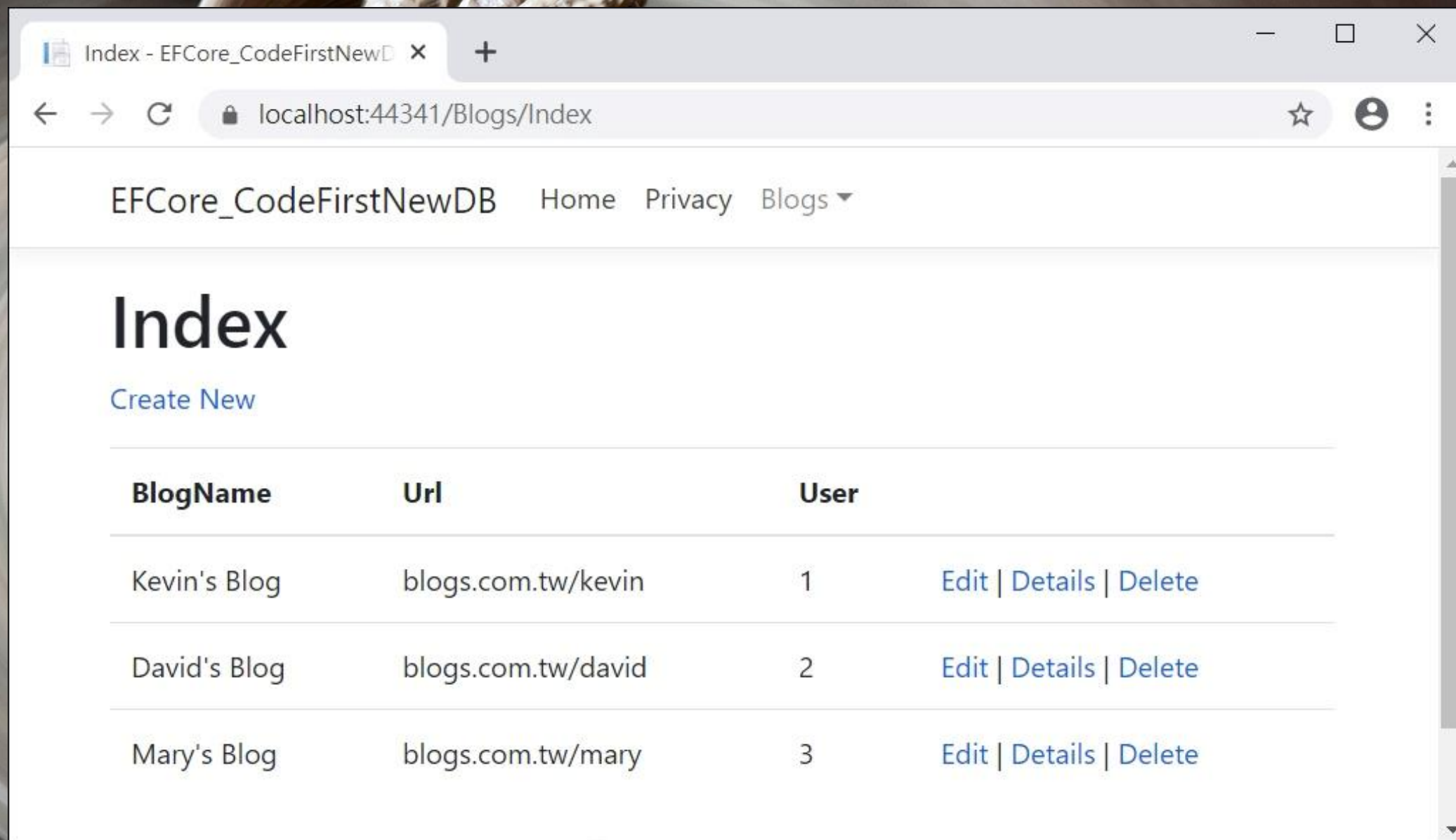
在 MVC 專案中建立過程(1)

- ① 建立 Entity 類別模型及 Entities 之間的 Navigation Property 及 Association 關聯設定
- ② 建立 DbContext 的衍生類別 BlogContext，並宣告公開的 DbSet<T>實體集
- ③ 在 appsettings.json 建立資料庫連線設定，供 BlogContext類別使用
- ④ 在 DI Container 中註冊 BlogContext 服務

在 MVC 專案中建立過程(2)

- ⑤ 以 Migrations 產生資料庫
- ⑥ 建立種子資料，用 Migrations 更新到資料庫
- ⑦ 以 Scaffolding 產出 Controller 及 Views 程式

範例 16-1 在MVC專案中 以Code First建立Blog應用程式及資料庫



Index - EFCore_CodeFirstNewDB

localhost:44341/Blogs/Index

EFCore_CodeFirstNewDB Home Privacy Blogs

Index

[Create New](#)

BlogName	Url	User	
Kevin's Blog	blogs.com.tw/kevin	1	Edit Details Delete
David's Blog	blogs.com.tw/david	2	Edit Details Delete
Mary's Blog	blogs.com.tw/mary	3	Edit Details Delete

DbContext 與 DbSet<TEntity> 功用

16-3

BlogContext 與 DbSet<TEntity>

繼承 DbContext

```
public class BlogContext : DbContext
{
    public BlogContext(DbContextOptions<BlogContext>
        options) : base(options)
    {
    }

    public virtual DbSet<User> Users { get; set; }
    public virtual DbSet<Blog> Blogs { get; set; }
    public virtual DbSet<Post> Posts { get; set; }
}
```

DbContext 選項

DbSet

DbContext 與 DbSet<TEntity> 功用

- **DbContext** 就是 EF 負責對資料庫作業的 Context 環境
- DbSet<TEntity> 則提供 Entity 實體集的 CRUD 讀寫等方法
- 有了 DbContext 及 DbSet<TEntity>，才能撰寫 EF 的 CRUD 程式

DbContextOptions 功用

- 設定 Database Provider，內建有 SQL Server、Sqlite、In-Memory和 Cosmos 四種提供者，詳細請參閱 <http://bit.ly/2TjvJy2>
- 設定資料庫連線
- Provider 層級選項行為
- EF Core 行為

EF名詞與關聯式資料庫對應

- 一個DbContext對應一個資料庫
- 一個DbSet對應一個Table
- 一個Entity對應一筆Row資料



請牢記

LINQ與DbSet查詢

- `DbSet<TEntity>` 是 Entity Set 實體集，一個 `DbSet<TEntity>` 中會包含許多 Entity 實體資料
- EF Core 的 LINQ 查詢是以 `DbSet<TEntity>` 為查詢對象
- EF Core 會在背景將 LINQ 轉為實際的 SQL 查詢語法
- 資料庫回傳的結果集就是儲存在 `DbSet<TEntity>` 中

DbContext初始化方式

DbContext 負責提供對資料庫作業所需環境及功能，因此對資料庫做讀寫前，須初始化DbContext物件：

- ① 使用 DI相依性注入
- ② 用 new 初始化

✦ 在 Controller 中以 DI 注入 BlogContext 實例

```
public class BlogsController : Controller
{
    private readonly BlogContext _context;
    public BlogsController(BlogContext context)
    {
        _context = context;
    }
}
```

以 DI 注入 BlogContext 實例

BlogContext 實例

✦ 用 new 初始化 BlogContext 實例

這種方式比較適合在無法使用 DI 或沒有 DI 的環境，例如 Console

```
using (var context = new BlogContext())
{
    //CRUD 語法
    ...
}
```

BlogContext 實例

Read查詢資料

```
using (var context = new NorthwindContext())  
{  
    var products = from p in context.Products ← DbSet<Product>  
                    select p;  
  
    ...  
}
```


Update更新資料

```
using (var context = new NorthwindContext())  
{  
    //以 find(id) 找尋資 Entity  
    var p = db.Products.FindAsync(64); ← DbSet<Product> 的 Find 方法  
    p.UnitsInStock = 13;  
    context.SaveChangesAsync(); //儲存變更  
}
```


Create新增資料

```
using (var context = new NorthwindContext())
{
    //新增一筆 Product Entity
    Product p = new Product { ProductName = "Car", UnitPrice = 100000,
        UnitsInStock = 1, UnitsOnOrder = 10 };
    //用 Add() 方法將 Entity 加入到 Products
    db.Products.Add(p); ← DbSet<Product>的 Add 方法
    //呼叫 SaveChangesAsync() 儲存變更時,新增至資料庫
    context.SaveChangesAsync();
}
```


Delete刪除資料

```
using (var context = new NorthwindContext())
{
    //用 id 找尋 Entity
    var p = db.Products.FindAsync(10);
    //用 Remove() 將 Entity 標記為刪除,
    db.Products.Remove(p); ← DbSet<Product>的 Remove 方法
    context.SaveChangesAsync();
}
```


DbContext調用與DbContextOptions 設定資料庫Provider及連線

16-4

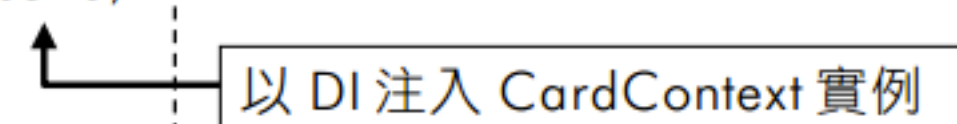
DbContext 調用的幾種方式

16-4-1

1. 透過DI相依性注入

```
public class CardsController : Controller
{
    //在 DI Container 註冊 CardContext 服務時,
    //需用 DbContextOptions 指定 Provider&資料庫連線, 才能用此方式注入
    private readonly CardContext _context;
    public CardsController(CardContext context)
    {
        _context = context;
    }

    //使用 CardContext 實例讀取 Cards 資料表記錄
    public async Task<IActionResult> CardListByDI()
    {
        List<Card> cards = await _context.Cards.AsNoTracking().ToListAsync();
        return View(cards);
    }
}
```



A diagram illustrating the DI injection process. A dashed rectangular box encloses the constructor of the `CardsController` class, specifically the `public CardsController(CardContext context)` method and the assignment `_context = context;`. An arrow points from a text box on the right, labeled "以 DI 注入 CardContext 實例", to the `context` parameter in the constructor signature.

2. 透過IServiceProvider介面的 GetService()或GetRequiredService()方法

- GetService 和 GetRequiredService 方法的差異，在於取得的服務若不存在，前者會回傳 null，後者則擲出例外
- 但在服務正常情況下，二者回傳是沒有差異的



Controllers/CardsController.cs

```
public async Task<IActionResult> CardContextByIServiceProvider([FromServices]
    IServiceProvider sp)
{
    var _context = sp.GetService<CardContext>();
    //var _context = sp.GetRequiredService<CardContext>();

    List<Card> cards = await _context.Cards.AsNoTracking().ToListAsync();
    return View(cards);
}
```

↑ 注入 IServiceProvider 實例

↓ 從 IServiceProvider 取得 CardContext 服務

直接初始化CardContext類別

```
using (var context = new CardContext())  
{  
    ...  
}
```

MS - 使用 'new' 的簡單 DbContext 初始化

<https://bit.ly/3CII0YZ>

用DbContextOptionBuilder及 DbContextOptions設定資料庫Provider與連線

16-4-2

在 Startup 中註冊DbContext時就一併指定資料庫 Provider 與連線字串

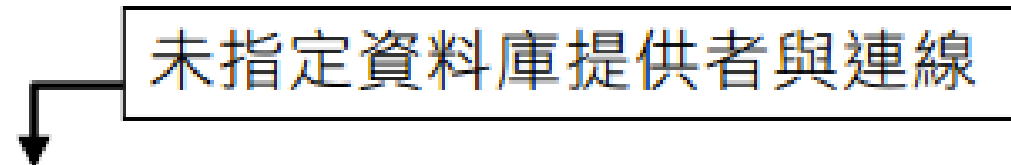
```
public void ConfigureServices(IServiceCollection services)
{
    services.AddDbContext<CardContext>(options =>
        options.UseSqlServer(Configuration.GetConnectionString("CardSqlServerDB")) ;
}
```

指定資料庫連線

指定 SQL Server 提供者

是否能只註冊 CardContext 服務，而不指定資料庫 Provider 與連線，留待執行時，由程式動態指定？

```
public void ConfigureServices(IServiceCollection services)
{
    ...
    services.AddDbContext<CardContext>();
}
```



未指定資料庫提供者與連線

用DbContextOptionsBuilder 達成以上需求

Controllers/CardsController.cs

```
public async Task<IActionResult> CardListByOptionsBuilder()
{
    var optionsBuilder = new DbContextOptionsBuilder<CardContext>();
    //設定:1.SQL Server Provider , 2.資料庫連線

    optionsBuilder.UseSqlServer(_config.GetConnectionString("CardSqlServerDB"));

    List<Card> cards = new List<Card>();
    //將 DbContextOptionsBuilder 傳入到 CardContext 建構函式
    using (CardContext ctx = new CardContext(optionsBuilder.Options))
    {
        cards = await ctx.Cards.AsNoTracking().ToListAsync();
    }

    return View(cards);
}
```

1. 建立 DbContextOptionsBuilder

2. 設定 Provider 及資料庫連線

3. 將 DbContextOptions 傳給 CardContext

用OnConfiguring 方法設定

 Models/CardContext.cs

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlServer(_config.GetConnectionString("CardSqlServerDB"));
}
```


使用 Sqlite、MySQL 及 In-Memory 提供者跨資料庫平台

16-5

表 16-2 EF Core 資料庫提供者

NuGet 套件	支援的資料庫引擎	維護者/廠商
Microsoft.EntityFrameworkCore.SqlServer	SQL Server 2012 及更新版本	EF Core 專案 (Microsoft)
Microsoft.EntityFrameworkCore.Sqlite	SQLite 3.7 及更新版本	EF Core 專案 (Microsoft)
Microsoft.EntityFrameworkCore.InMemory	EF Core 記憶體 內部資料庫	EF Core 專案 (Microsoft)
Microsoft.EntityFrameworkCore.Cosmos	Azure Cosmos DB SQL API	EF Core 專案 (Microsoft)
Npgsql.EntityFrameworkCore.PostgreSQL	PostgreSQL	Npgsql 開發小組
Pomelo.EntityFrameworkCore.MySql	MySQL、MariaDB	Pomelo Foundation 專案
MySql.Data.EntityFrameworkCore	MySQL	MySQL project (Oracle)
Oracle.EntityFrameworkCore	Oracle DB 11.2 與更新版本	Oracle

EF Core 資料庫提供者詳細列表<http://bit.ly/2TjvJy2>

EF Core 針對不同資料庫平台 作存取三步驟

- 安裝特定平台資料庫提供者 NuGet 套件
- 在 appsettings.json 建立不同平台資料庫連線設定
- 透過 DbContextOptionsBuilder 指定對應的資料庫提供者與連線

一、安裝資料庫提供者NuGet 套件

✦ 以 Package Manager Console 命令：

```
Install-package Microsoft.EntityFrameworkCore.SqlServer  
Install-package Microsoft.EntityFrameworkCore.Sqlite  
Install-package Microsoft.EntityFrameworkCore.InMemory  
Install-package Pomelo.EntityFrameworkCore.MySql
```

✦ 以 .NET Core CLI 命令

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer  
dotnet add package Microsoft.EntityFrameworkCore.Sqlite  
dotnet add package Microsoft.EntityFrameworkCore.InMemory  
dotnet add package Pomelo.EntityFrameworkCore.MySql
```


二、在 appsettings.json 建立不同平台資料庫連線設定

- 建立 SQL Server、Sqlite 和 MySQL 三種資料庫連線



appsettings.json

```
{  
  ...,  
  "ConnectionStrings": {  
    "CardSqlServerDB": "Server=(localdb)\\mssqllocaldb;Database=CardSqlServerDB;  
                        Trusted_Connection=True;MultipleActiveResultSets=true",  
    "CardSqliteDB": "Data Source = CardSqliteDB.db",  
    "CardMySqlDB": "server=localhost;database=CardMySqlDB;  
                  user=root;password=yourpassword"  
  }  
}
```

SQL Server 連線

Sqlite 連線

MySQL 連線

三、透過 DbContextOptionsBuilder 指定對應的資料庫提供者與連線

- 以下擇一種方式註冊

```
public void ConfigureServices(IServiceCollection services)
{
    //請擇一註冊使用
    services.AddDbContext<CardContext>(options =>
        options.UseSqlServer(Configuration.GetConnectionString("CardSqlServerDB")));
    services.AddDbContext<CardContext>(options =>
        options.UseSqlite(Configuration.GetConnectionString("CardSqliteDB")));
    services.AddDbContext<CardContext>(options =>
        options.UseMySQL(Configuration.GetConnectionString("CardMySQLDB")));
    services.AddDbContext<CardContext>(options =>
        options.UseInMemoryDatabase("InMemoryDB"));
}
```


EF Core 的 CLI 命令工具

16-6

EF Core 的 CLI 命令工具

EF Core 的命令工具分為兩類：

- Visual Studio - PackageManager Console (PMC) 中的命令，適合在 Visual Studio 中使用
- .NET Core SDK CLI 命令，適合在沒有 Visual Studio 或命令視窗中使用

CLI與PMC命令對照表

表 16-3 EF Core 的 CLI 命令

EF Core 的 CLI 命令	PMC 命令
dotnet ef migrations add	add-migration
dotnet ef migrations remove	remove-migration
dotnet ef migrations list	--
dotnet ef migrations script	script-migration
dotnet ef database update	update-database
dotnet ef database drop	drop-database
dotnet ef dbcontext info	get-dbcontext -context <DbContext 名稱>
dotnet ef dbcontext list	get-dbcontext
dotnet ef dbcontext scaffold	Scaffold-DbContext

安裝 EF Core CLI 命令所需環境

16-6-1

安裝 EF Core CLI 命令工具

- 安裝

dotnet tool **install** --global dotnet-ef

- 查看版本

dotnet tool **list** --global

- 更新

dotnet tool **update** --global dotnet-ef

- 移除

dotnet tool **uninstall** —global dotnet-ef

安裝EF Core相關NuGet套件

▪ 在命令視窗安裝的.NET CLI 命令

```
dotnet add package Microsoft.EntityFrameworkCore.Tools  
dotnet add package Microsoft.EntityFrameworkCore.SqlServer  
dotnet add package Microsoft.EntityFrameworkCore.Design  
dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design
```

▪ 在 NuGet 套件管理器主控台安裝的命令

```
Install-Package Microsoft.EntityFrameworkCore.Tools (Migrations 需此套件)  
Install-package Microsoft.EntityFrameworkCore.SqlServer (SQL Server Provider)  
Install-Package Microsoft.EntityFrameworkCore.Design  
Install-Package Microsoft.VisualStudio.Web.CodeGeneration.Design
```


EF Core 的 CLI 命令工具用法

16-6-2

dotnet ef migrations add 建立新的 Migration 檔

- 當 DbContext 或 Model 模型有異動時，須以此命令新增一個 Migration 檔
- 其用意在於提交一個異動，然後在更新資料庫時，會套用這個 Migration 異動檔

dotnet ef **migration add InitialDB**

dotnet ef migrations remove

移除最新的Migration

- Migration 未更新至資料庫

```
dotnet ef migrations remove
```

- Migration 已更新到資料庫，則須使用 -f 或 --force 參數強制移除，並再次執行更新資料庫

```
dotnet ef migrations remove --force
```

```
dotnet ef database update
```


dotnet ef database update 更新資料庫

- 將最新 Migration 更新到資料庫

dotnet ef **database update**

- 指定 Migration 版本更新，指定更新可用 Migration 名稱或 ID

dotnet ef database update **InitialDB**(指定名稱)

dotnet ef database update

20200220123934_InitialDB(指定Id)

dotnet ef database drop

移除資料庫

- 此命令移除資料庫前會先詢問，輸入 y 之後，便會移除資料庫

dotnet ef database **drop**

輸出：

```
Are you sure you want to drop the database 'CoreMvc_BlogDB' on server  
'(localdb)\mssqllocaldb'? (y/N)  
y  
Dropping database 'CoreMvc_BlogDB'.  
Successfully dropped database 'CoreMvc_BlogDB'.
```


dotnet ef dbcontext list

列出專案所有DbContext名稱

dotnet ef **dbcontext list** (處於 Project 目錄)

dotnet ef **dbcontext list -p EFCore_CodeFirstNewDB**

dotnet ef **dbcontext list --project
EFCore_CodeFirstNewDB**

輸出：

```
EFCore_CodeFirstNewDB.Models.BlogContext  
EFCore_CodeFirstNewDB.Models.CardContext
```


若有多個 DbContext

- Migration 都會依附特定 DbContext，可用 **-c** 或 **--context** 參數

dotnet ef migrations list **-c BlogContext**

dotnet ef migrations list **--context BlogContext**

輸出：

```
20200220123934_InitialDB  
20200220125527_AddSeedData
```


dotnet ef dbcontext info 顯示 DbContext 詳細資訊

dotnet ef **dbcontext info**

dotnet ef **dbcontext info -c BlogContext**

dotnet ef **dbcontext info --context BlogContext**

輸出：

```
Provider name: Microsoft.EntityFrameworkCore.SqlServer  
Database name: CardSqlServerDB  
Data source: (localdb)\mssqllocaldb  
Options: None
```


dotnet ef migrations script 從 Migrations 產生資料庫SQL Script

- 語法：dotnet ef **migrations script** **<FROM>** **<TO>**

dotnet ef migrations script (從 0 至最後一個 Migration)

dotnet ef migrations script 0 AddSeedData (從 0 之後開始，到最後一個)

dotnet ef migrations script 0 InitialDB (從 0 之後開始，到第一個)

dotnet ef migrations script InitialDB AddSeedData (從第一個之後開始，到第二個)

dotnet ef migrations script InitialDB (從第一個之後開始，第一個忽略不計)

dotnet ef migrations script InitialDB -o SQL/SeedDataScript (輸出至特定目錄)

dotnet ef dbcontext scaffold 產出DbContext及模型檔

- 對 Northwind 資料庫產出 DbContext 及所有Model模型檔

```
dotnet ef dbcontext scaffold "Data Source=(localdb)\mssqllocaldb;Database=Northwind;"  
Microsoft.EntityFrameworkCore.SqlServer -o Models
```

- 用-t參數指定資料表產出所需 Model 檔

```
dotnet ef dbcontext scaffold Name=NorthwindConnection  
Microsoft.EntityFrameworkCore.SqlServer -o Models  
-t Orders -t "Order Details" -t Products -t Categories  
-t Suppliers -t Employees -t Customers -t Shippers
```


The End

奚江華個人經歷簡介

- 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹**、**ASP.NET Core 3.x MVC跨平台範例實戰演練**
- 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- 曾數屆當選微軟MVP
- 國內各大上市公司及大專院校之培訓講師
- 曾發表文章刊於ITHOME電腦週刊

碼魔法FB

- CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw